`ranges::fold`

Document #:	P2322R0
Date:	2021-02-16
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

As described in [P2214R0], there is one very important rangified algorithm missing from the standard library: `fold`.

While we do have an iterator-based version of `fold` in the standard library, it is currently named `accumulate`, defaults to performing `+` on its operands, and is found in the header `<numeric>`. But `fold` is much more than addition, so as described in the linked paper, it's important to give it the more generic name and to avoid a default operator.

Also as described in the linked paper, it is important to avoid over-constraining `fold` in a way that prevents using it for heterogeneous folds. As such, the `fold` specified in this paper only requires one particular invocation of the binary operator and there is no `common_reference` requirement between any of the types involved.

Lastly, the `fold` here is proposed to go into `<algorithm>` rather than `<numeric>` since there is nothing especially numeric about it.

§ 2 Other `fold` algorithms

[P2214R0] proposed a single fold algorithm that takes an initial value and a binary operation and performs a *left* fold over the range. But there are a couple variants that are also quite valuable and that we should adopt as a family.

§ 2.1 `fold_first`

Sometimes, there is no good choice for the initial value of the fold and you want to use the first element of the range. For instance, if I want to find the smallest string in a range, I can already do that as `ranges::min(r)` but the only way to express this in terms of `fold` is to manually pull out the first element, like so:

```
auto b = ranges::begin(r);
auto e = ranges::end(r);
ranges::fold(ranges::next(b), e, *b, ranges::min);
```

But this is both tedious to write, and subtly wrong for input ranges anyway since if the `next(b)` is evaluated before `*b`, we have a dangling iterator. This comes up enough that this paper proposes a version of `fold` that uses the first element in the range as the initial value (and thus has a precondition that the range is not empty).

This algorithm exists in Rust (under the name `fold_first` as a nightly-only experimental API and `fold1` in the `Itertools` crate) and Haskell (under the name `foldl1`). This paper proposes the name `fold_first` rather than deal with the question of if we can sufficiently constrain our overloads to avoid ambiguity. Plus, the fact that `fold` has no preconditions but `fold_first` does suggests that they should have different names.

§ 2.2 `fold_right`

While `ranges::fold` would be a left-fold, there is also occasionally the need for a *right*-fold. While a `fold_right` is much easier to write in code given `fold` than `fold_first`, since `fold_right(r, init, op)` is `fold(r | views::reverse, init, flip(op))`, it's sufficiently common that it may as well be in the standard library.

As with `fold_first`, we should also provide a `fold_right_last`.

There are three questions that would need to be asked about `fold_right`.

First, the order of operations of to the function. Given `fold_right([1, 2, 3], z, f)`, is the evaluation `f(1, f(2, f(3, z)))` or is the evaluation `f(f(f(z, 3), 2), 1)`? Note that either way, we're choosing the `3` then `2` then `1`, both are right folds. It's a question of if the initial element is the left-hand operand (as it is in the left `fold`) or the right-hand operand (as it would be if consider the right fold as a flip of the left fold). For instance, Scheme picks the former but Haskell picks the latter.

One advantage of the former is that we can specify `fold_right(r, z, op)` as precisely `fold_left(views::reverse(r), z, op)` and leave it at that. With the latter, we would need need slightly more specification and would want to avoid saying `flip(op)` since directly invoking the operation with the arguments in the correct order is a little better in the case of ranges of prvalues.

This paper picks the latter (that is `fold_right` as the order of arguments flipped from `fold`).

Second, supporting bidirectional ranges is straightforward. Supporting forward ranges involves recursion of the size of the range. Supporting input ranges involves recursion and also copying the whole range

first. Are either of these worth supporting? The paper simply supports bidirectional ranges.

Third, the naming question. Given that we have `fold_right`, should the other one be named `fold_left`? Or we could take Haskell's names of `foldl` and `foldr`? In my experience, left-folds are more common than right-folds, so this paper proposes the names `fold`/`fold_first` and `fold_right`/`fold_right_last`.

§ 3 Wording

Append to 25.4 [algorithm.syn]:

```
#include <initializer_list>

namespace std {
  // ...

  // [alg.fold], folds
  namespace ranges {
    template<class LHS, class RHS>
    concept weakly-assignable-from =          // exposition only
      requires(LHS lhs, RHS&& rhs) {
        { lhs = std::forward<RHS>(rhs); } -> same_as<LHS>;
      };

    template<class F, class R, class... Args>
    concept foldable =                        // exposition only
      movable<R> &&
      copy_constructible<F> &&
      regular_invocable<F&, Args...> &&
      weakly-assignable-from<R&, invoke_result_t<F&, Args...>>;

    template<class F, class T, class I>
    concept indirectly-binary-left-foldable = // exposition only
      indirectly_readable<I> &&
      foldable<F, T, T, iter_reference_t<I>>;

    template<class F, class T, class I>
    concept indirectly-binary-right-foldable = // exposition only
      indirectly_readable<I> &&
      foldable<F, T, iter_reference_t<I>, T>;

    template<input_iterator I, sentinel_for<I> S, class T, class Proj =
        identity,
      indirectly-binary-left-foldable<T, projected<I, Proj>>
        BinaryOperation>
    constexpr T fold(I first, S last, T init, BinaryOperation binary_op,
        Proj proj = {});

    template<input_range R, class T, class Proj = identity,
      indirectly-binary-left-foldable<T, projected<iterator_t<R>, Proj>>
        BinaryOperation>
    constexpr T fold(R&& r, T init, BinaryOperation binary_op, Proj proj =
        {});

    template <input_iterator I, sentinel_for<I> S, class Proj = identity,
```

```
      indirectly-binary-left-foldable<iter_value_t<I>, projected<I, Proj>>
        BinaryOperation>
      requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr iter_value_t<I> fold_first(I first, S last, BinaryOperation
        binary_op, Proj proj = {});

    template <input_range R, class Proj = identity,
      indirectly-binary-left-foldable<range_value_t<R>,
        projected<iterator_t<R>, Proj>> BinaryOperation>
      requires constructible_from<range_value_t<I>, range_reference_t<I>>
    constexpr range_value_t<R> fold_first(R&& r, BinaryOperation binary_op,
        Proj proj = {});

    template<bidirectional_iterator I, sentinel_for<I> S, class T, class
        Proj = identity,
      indirectly-binary-right-foldable<T, projected<I, Proj>>
        BinaryOperation>
    constexpr T fold_right(I first, S last, T init, BinaryOperation
        binary_op, Proj proj = {});

    template<bidirectional_range R, class T, class Proj = identity,
      indirectly-binary-right-foldable<T, projected<iterator_t<R>, Proj>>
        BinaryOperation>
    constexpr T fold_right(R&& r, T init, BinaryOperation binary_op, Proj
        proj = {});

    template <bidirectional_iterator I, sentinel_for<I> S, class Proj =
        identity,
      indirectly-binary-right-foldable<iter_value_t<I>, projected<I, Proj>>
        BinaryOperation>
      requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr iter_value_t<I> fold_right_last(I first, S last,
        BinaryOperation binary_op, Proj proj = {});

    template <bidirectional_range R, class Proj = identity,
      indirectly-binary-right-foldable<range_value_t<R>,
        projected<iterator_t<R>, Proj>> BinaryOperation>
      requires constructible_from<range_value_t<I>, range_reference_t<I>>
    constexpr range_value_t<R> fold_right_last(R&& r, BinaryOperation
        binary_op, Proj proj = {});
  }
}
```

And add a new clause, [alg.fold]:

```
template<input_iterator I, sentinel_for<I> S, class T, class Proj =
        identity,
  indirectly-binary-left-foldable<T, projected<I, Proj>> BinaryOperation>
constexpr T ranges::fold(I first, S last, T init, BinaryOperation binary_op,
        Proj proj = {});

template<input_range R, class T, class Proj = identity,
  indirectly-binary-left-foldable<T, projected<iterator_t<R>, Proj>>
        BinaryOperation>
constexpr T ranges::fold(R&& r, T init, BinaryOperation binary_op, Proj proj
        = {});
```

1 *Effects*: Equivalent to:

```
    while (first != last) {
        init = invoke(binary_op, std::move(init), invoke(proj,
            *first));
        ++first;
    }
    return init;
```

```
template <input_iterator I, sentinel_for<I> S, class Proj = identity,
  indirectly-binary-left-foldable<iter_value_t<I>, projected<I, Proj>>
        BinaryOperation>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr iter_value_t<I> ranges::fold_first(I first, S last,
        BinaryOperation binary_op, Proj proj = {});

template <input_range R, class Proj = identity,
  indirectly-binary-left-foldable<range_value_t<R>, projected<iterator_t<R>,
        Proj>> BinaryOperation>
  requires constructible_from<range_value_t<I>, range_reference_t<I>>
constexpr range_value_t<R> ranges::fold_first(R&& r, BinaryOperation
        binary_op, Proj proj = {});
```

2 *Preconditions*: `first != last` is `true`.

3 *Effects*: Equivalent to `return ranges::fold(ranges::next(std::move(first)), std::move(last), iter_value_t<I>(*first), binary_op, proj)` except ensuring that the initial value is constructed before the iterator is incremented if `first` is an input iterator.

```
template<bidirectional_iterator I, sentinel_for<I> S, class T, class Proj =
        identity,
  indirectly-binary-right-foldable<T, projected<I, Proj>> BinaryOperation>
constexpr T ranges::fold_right(I first, S last, T init, BinaryOperation
        binary_op, Proj proj = {});

template<bidirectional_range R, class T, class Proj = identity,
  indirectly-binary-right-foldable<T, projected<iterator_t<R>, Proj>>
        BinaryOperation>
constexpr T ranges::fold_right(R&& r, T init, BinaryOperation binary_op,
        Proj proj = {});
```

4 *Effects*: Equivalent to:

```
    I tail = ranges::next(first, last);
    while (first != tail) {
        init = invoke(binary_op, invoke(proj, *--tail),
            std::move(init));
    }
    return init;
```

```
template <bidirectional_iterator I, sentinel_for<I> S, class Proj =
        identity,
  indirectly-binary-right-foldable<iter_value_t<I>, projected<I, Proj>>
        BinaryOperation>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr iter_value_t<I> fold_right_last(I first, S last, BinaryOperation
        binary_op, Proj proj = {});

template <bidirectional_range R, class Proj = identity,
```

```
  indirectly-binary-right-foldable<range_value_t<R>,
        projected<iterator_t<R>, Proj>> BinaryOperation>
  requires constructible_from<range_value_t<I>, range_reference_t<I>>
constexpr range_value_t<R> fold_right_last(R&& r, BinaryOperation binary_op,
        Proj proj = {});
```

5 *Preconditions*: `first != last` is `true`.

6 *Effects*: Equivalent to:

```
I tail = ranges::prev(ranges::next(first, std::move(last));
return ranges::fold_right(std::move(first), tail, iter_value_t<I>
        (*tail), binary_op, proj);
```

§ 4 References

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges. https://wg21.link/p2214r0